

Nova OS Architekturhandbuch

Version: 0.1

Status: Entwurf

Sprache: Deutsch

1. Ziel des Architekturhandbuchs

Dieses Dokument definiert die technischen Grundregeln für Nova OS.

Es beschreibt:

- Projektstruktur
- Namenskonventionen
- Modulaufbau
- Codierstandard
- Fehlerbehandlung
- Logging
- Speicherregeln
- Sicherheitsprinzipien
- Grafikarchitektur
- Buildsystem
- Testregeln

Ziel ist, dass Nova OS auch bei tausenden Dateien wartbar, erweiterbar und verständlich bleibt.

2. Grundprinzipien

Nova OS folgt diesen Prinzipien:

1. Klarheit vor Cleverness
2. Stabilität vor Geschwindigkeit
3. Modularität vor Monolith
4. Sicherheit vor Komfort
5. Datenschutz vor Cloud
6. Lokale Verarbeitung vor Online-Diensten
7. Lesbarer Code vor kurzer Code
8. Dokumentierte Schnittstellen vor implizitem Verhalten
9. Keine versteckten Abhängigkeiten
10. Keine magischen Zahlen

3. Hauptstruktur

```
NovaOS/  
├─ boot/
```

```
|— kernel/
|— runtime/
|— libraries/
|— drivers/
|— services/
|— system/
|— recovery/
|— installer/
|— sdk/
|— applications/
|— assets/
|— themes/
|— tools/
|— tests/
└— docs/
```

4. Kernel-Regel

Der Kernel bleibt klein.

In den Kernel gehören nur:

- Speicherverwaltung
- Scheduler
- Interrupts
- Prozesse
- Threads
- IPC
- System Calls
- Sicherheitskern
- Treiberbasis
- Hardwareabstraktion

Nicht in den Kernel gehören:

- Desktop
- App Store
- Dateimanager
- KI-Assistent
- Update-Oberfläche
- Widgets
- normale Anwendungen

Diese laufen später als Dienste oder Anwendungen im Userspace.

5. Dateinamen

Alle Nova-Dateien nutzen das Präfix `nova_`.

Beispiele:

```
nova_kernel.c
nova_kernel.h
nova_logger.c
nova_logger.h
nova_framebuffer.c
nova_framebuffer.h
nova_scheduler.c
nova_scheduler.h
```

Keine generischen Namen wie:

```
main.c
utils.c
helper.c
test.c
```

Ausnahme:

```
kmain.c
```

für den Kernel-Einstieg.

6. Funktionsnamen

C-Funktionen nutzen dieses Schema:

```
nova_modul_aktion()
```

Beispiele:

```
nova_logger_info()
nova_framebuffer_init()
nova_scheduler_start()
nova_memory_alloc_page()
```

Keine unklaren Namen wie:

```
init()
start()
run()
handle()
doStuff()
```

7. Kommentare

Jede Datei beginnt mit einem Kopfkomentar:

```
/
*****
*
* Nova OS
*
* Datei:
*     nova_framebuffer.c
*
* Beschreibung:
*     Verwaltet den Framebuffer des Kernels.
*
* Hinweise:
*     Direkte Framebuffer-Zugriffe außerhalb dieses Moduls sind verboten.
*
*****/
```

Jede wichtige Funktion erhält eine Beschreibung.

Kommentare erklären nicht nur **was**, sondern auch **warum**.

8. Fehlermeldungen

Alle sichtbaren Fehlermeldungen sind auf Deutsch.

Beispiele:

```
[FEHLER] Framebuffer konnte nicht initialisiert werden.
[WARNUNG] Kein Mausgerät gefunden. Tastatursteuerung wird verwendet.
[INFO] Kernel wurde erfolgreich gestartet.
```

Interne Fehlercodes dürfen englische Konstanten nutzen.

9. Logging-Standard

Log-Level:

```
DEBUG
INFO
WARNUNG
FEHLER
KRITISCH
ERFOLG
```

Format:

```
[NOVA][INFO] Speicherverwaltung gestartet.
[NOVA][FEHLER] Kernelmodul konnte nicht geladen werden.
```

10. Speicherregeln

Direkter Speicherzugriff ist nur in klar definierten Modulen erlaubt.

Erlaubt in:

- memory/
- framebuffer/
- paging/
- dma/
- drivers/

Nicht erlaubt in:

- UI
- normalen Diensten
- Anwendungen
- High-Level-Bibliotheken

11. Grafikarchitektur

Nova OS verwendet die **Nova Graphics Engine (NGE)**.

Regeln:

- Keine festen Auflösungen.
- Keine festen Koordinaten ohne View-System.
- Alle UI-Elemente nutzen adaptive Skalierung.
- Rendering erfolgt über Buffer.

- Direkter Framebuffer-Zugriff nur im Framebuffer-Modul.
- Später Unterstützung für GPU-Backends.

Pflichtkomponenten:

```
Framebuffer Manager
Buffer Manager
View System
Primitive Renderer
Text Renderer
Image Renderer
Effects Engine
Compositor
Theme Engine
```

12. Adaptives View-System

Alle Oberflächen müssen sich automatisch anpassen an:

- Querformat
- Hochformat
- Ultra-Wide
- kleine Displays
- große Displays
- hohe DPI
- Touch
- Maus
- Tastatur

Referenzauflösung:

```
1920 × 1080
```

Alle UI-Größen werden relativ skaliert.

13. Sicherheit

Nova OS folgt dem Prinzip:

```
Standardmäßig sicher.
Optional offen.
Niemals heimlich.
```

Regeln:

- Keine Cloud ohne Zustimmung.
- Keine Telemetrie ohne Zustimmung.
- Keine versteckten Hintergrunddienste.
- Signierte Systemkomponenten.
- Optionales Boot-Passwort.
- Optional TPM.
- Optional vollständige Verschlüsselung.
- Recovery-Menü kann geschützt werden.

14. Bootprozess

Zielarchitektur:

```
BIOS / UEFI
  ↓
Nova Boot Manager
  ↓
Nova Recovery Environment oder Kernel
  ↓
Nova Kernel
  ↓
Systemdienste
  ↓
Desktop
```

Der Bootloader bleibt klein.

Das grafische Recovery-System übernimmt komplexe Aufgaben wie:

- Backup
- Wiederherstellung
- Selbstheilung
- Speichertest
- Formatierung
- Ver- und Entschlüsselung

15. Buildsystem

Standard:

```
CMake
```

Ziele:

```
nova_bootloader_uefi  
nova_bootloader_bios  
nova_kernel  
nova_recovery  
nova_image
```

Jedes Modul besitzt eine eigene `CMakeLists.txt`.

16. Tests

Jedes Modul soll testbar sein.

Testarten:

- Unit-Test
- Integrationstest
- Emulator-Test
- QEMU-Test
- Boot-Test
- Regressionstest

Kein Modul gilt als fertig, wenn es nicht mindestens einmal isoliert getestet wurde.

17. Dokumentationsregel

Zu jedem größeren Modul gehören:

```
README.md  
ARCHITECTURE.md  
API.md  
TESTS.md
```

18. Versionsregel

Nova OS nutzt semantische Versionierung:

```
MAJOR.MINOR.PATCH
```

Beispiel:

```
0.1.0
```


Solange Nova OS nicht stabil ist, bleibt es unter Version `1.0.0`.

19. Aktueller Entwicklungsstand

Aktuell definiert:

- Projektstruktur
 - Bootloader-Konzept
 - UEFI-Einstieg
 - Bootmenü-Konzept
 - Kernel Entry
 - Boot Context
 - Framebuffer Manager
 - Nova Graphics Engine
 - Adaptives View-System
 - Architekturhandbuch
-

20. Nächster Schritt

Als nächstes werden die vorhandenen Dateien konsolidiert und nach diesem Handbuch neu ausgegeben.

Reihenfolge:

1. Projektstruktur
2. Root-Dateien
3. Bootloader-Dateien
4. Kernel-Dateien
5. Grafiksystem-Dateien
6. Buildsystem